Ex8: Interactive Dashboards – Sample Superstore

Dataset: Sample – Superstore (Kaggle) – `Sample - Superstore.csv`

The Tableau steps of this experiment (worksheets, dashboard, filters, actions and export) are reproduced in Python so they can be run in Colab. Upload `Sample - Superstore.csv` to the Colab **Files** panel before running.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

sns.set_style('whitegrid')
pd.set_option('display.max_columns', None)

REGION_COLORS = {'Central': '#4E79A7', 'East': '#F28E2B', 'South': '#E15759', 'West': '#76B7B2'}
CATEGORY_COLORS = {'Furniture': '#4E79A7', 'Office Supplies': '#F28E2B', 'Technology': '#59A14F'}
SEGMENT_MARKERS = {'Consumer': 'o', 'Corporate': 's', 'Home Office': '^'}
currency = mticker.FuncFormatter(lambda x, _: f'${x:,.0f}')
currency_k = mticker.FuncFormatter(lambda x, _: f'${x / 1000:,.0f}K')
```

Task 1: Load the dataset

```
try:
    df = pd.read_csv('Sample - Superstore.csv')
except UnicodeDecodeError:                          # the original Kaggle file is Windows-1252 encoded
    df = pd.read_csv('Sample - Superstore.csv', encoding='latin1')

df['Order Date'] = pd.to_datetime(df['Order Date'], format='mixed')
df['Ship Date'] = pd.to_datetime(df['Ship Date'], format='mixed')
df['Year'] = df['Order Date'].dt.year
print("Dataset loaded:", df.shape)
```

```
Dataset loaded: (9994, 22)
```

Task 2: Explore the Data

```
display(df.head())
print("Shape:", df.shape)
print("\nData types:")
print(df.dtypes)
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country
0	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States
1	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States
2	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States
3	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States
4	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States

	City	State	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales
0	Henderson	Kentucky	42420	South	FUR-BO-10001798	Furniture	Bookcases	Bush Somerset Collection Bookcase	261.9600
1	Henderson	Kentucky	42420	South	FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400
2	Los Angeles	California	90036	West	OFF-LA-10000240	Office Supplies	Labels	Self-Adhesive Address Labels for Typewriters b...	14.6200
3	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577	Furniture	Tables	Bretford CR4500 Series Slim Rectangular Table	957.5775
4	Fort Lauderdale	Florida	33311	South	OFF-ST-10000760	Office Supplies	Storage	Eldon Fold 'N Roll Cart System	22.3680

	Quantity	Discount	Profit	Year
0	2	0.00	41.9136	2016
1	3	0.00	219.5820	2016
2	2	0.00	6.8714	2016
3	5	0.45	-383.0310	2015
4	2	0.20	2.5164	2015

```
Shape: (9994, 22)

Data types:
Row ID                    int64
Order ID                    str
Order Date       datetime64[us]
Ship Date        datetime64[us]
Ship Mode                   str
Customer ID                 str
Customer Name               str
Segment                     str
Country                     str
City                        str
State                       str
Postal Code               int64
Region                      str
Product ID                  str
Category                    str
Sub-Category                str
Product Name                str
Sales                   float64
Quantity                  int64
Discount                float64
Profit                  float64
Year                      int32
dtype: object
```

```
print("Missing values:", df.isnull().sum().sum())
print("Duplicate records:", df.duplicated().sum())
print("Order date range:", df['Order Date'].min().date(), "to", df['Order Date'].max().date())
display(df[['Sales', 'Quantity', 'Discount', 'Profit']].describe().round(2).T)

for col in ['Category', 'Sub-Category', 'Region', 'Segment']:
    print(f"{col} ({df[col].nunique()}): {sorted(df[col].unique())}")

print(f"\nTotal Sales : ${df['Sales'].sum():,.2f}")
print(f"Total Profit: ${df['Profit'].sum():,.2f}")
```

```
Missing values: 0
Duplicate records: 0
Order date range: 2014-01-03 to 2017-12-30
```

	count	mean	std	min	25%	50%	75%	max
Sales	9994.0	229.86	623.25	0.44	17.28	54.49	209.94	22638.48
Quantity	9994.0	3.79	2.23	1.00	2.00	3.00	5.00	14.00
Discount	9994.0	0.16	0.21	0.00	0.00	0.20	0.20	0.80
Profit	9994.0	28.66	234.26	-6599.98	1.73	8.67	29.36	8399.98

```
Category (3): ['Furniture', 'Office Supplies', 'Technology']
Sub-Category (17): ['Accessories', 'Appliances', 'Art', 'Binders', 'Bookcases', 'Chairs', 'Copiers', 'Envelopes', 'Fasteners',
'Furnishings', 'Labels', 'Machines', 'Paper', 'Phones', 'Storage', 'Supplies', 'Tables']
Region (4): ['Central', 'East', 'South', 'West']
Segment (3): ['Consumer', 'Corporate', 'Home Office']

Total Sales : $2,297,200.86
Total Profit: $286,397.02
```

Worksheet functions

Each function draws one Tableau worksheet. `data` is the (filtered) dataset, so the same functions are reused for the dashboard, the filters and the actions.

```
def bar_sales_by_category(data, ax, highlight_category=None):
    """Worksheet 1: Category on Columns, SUM(Sales) on Rows, Region on Color, sorted descending, mark labels."""
    pivot = data.pivot_table(index='Category', columns='Region', values='Sales', aggfunc='sum', fill_value=0)
    pivot = pivot.loc[pivot.sum(axis=1).sort_values(ascending=False).index]      # sort bars descending
    bottom = np.zeros(len(pivot))
    for region in pivot.columns:
        alpha = [1.0 if highlight_category in (None, c) else 0.25 for c in pivot.index]
        bars = ax.bar(pivot.index, pivot[region], bottom=bottom, color=REGION_COLORS[region],
                      edgecolor='white', label=region)
        for bar, a in zip(bars, alpha):
            bar.set_alpha(a)
        ax.bar_label(bars, labels=[f'${v/1000:,.0f}K' if v > 0 else '' for v in pivot[region]],
                     label_type='center', fontsize=8, color='white', fontweight='bold')
        bottom += pivot[region].values
    for i, total in enumerate(bottom):
        ax.text(i, total, f'${total:,.0f}', ha='center', va='bottom', fontsize=9, fontweight='bold')
    ax.yaxis.set_major_formatter(currency)
    ax.set_title('Bar_Sales_by_Category')
    ax.set_xlabel('Category')
    ax.set_ylabel('SUM(Sales)')
    ax.legend(title='Region', fontsize=7, title_fontsize=8, ncol=4, loc='upper center')
    ax.set_ylim(0, bottom.max() * 1.35)
```

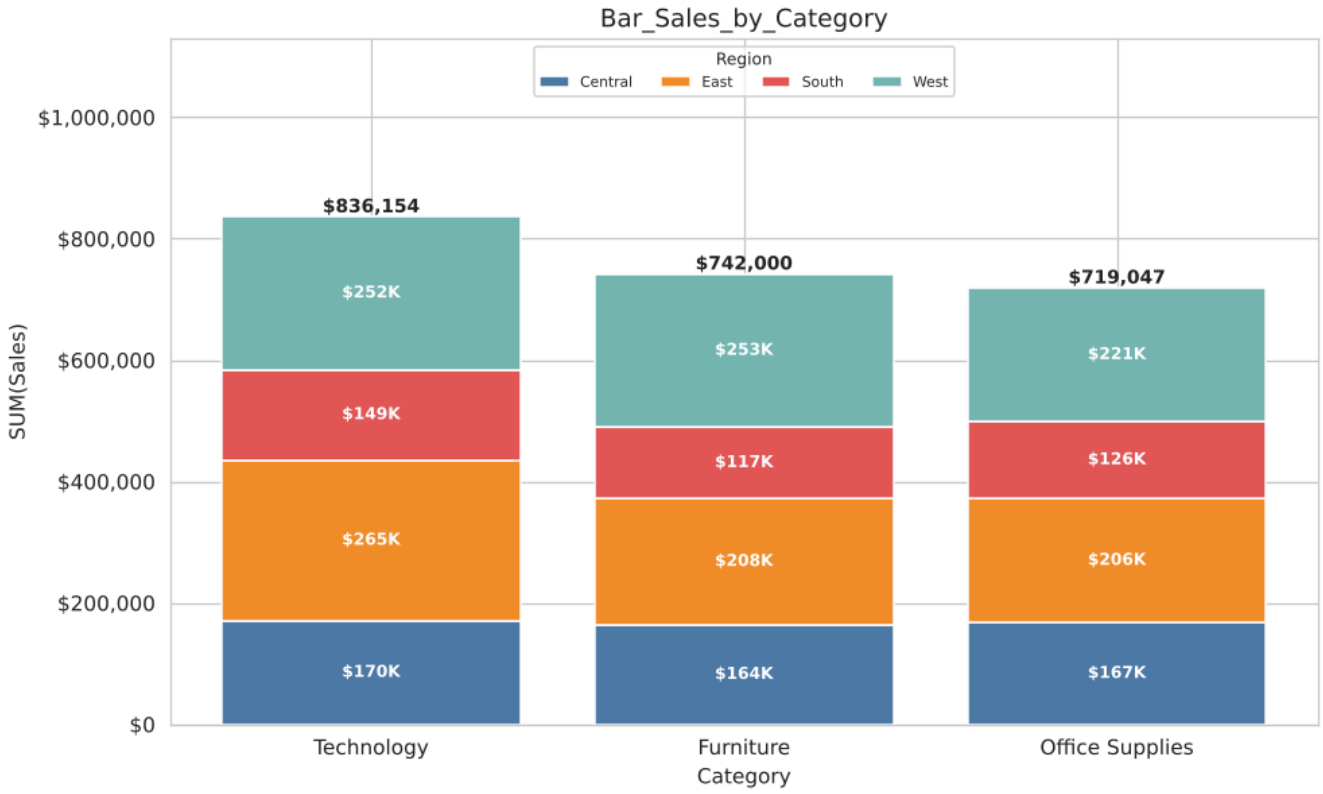
```
def line_sales_trend(data, ax):
    """Worksheet 2: MONTH(Order Date) continuous on Columns, SUM(Sales) on Rows, Region on Color."""
    monthly = (data.groupby([pd.Grouper(key='Order Date', freq='MS'), 'Region'])['Sales']
               .sum().unstack(fill_value=0))
    for region in monthly.columns:
        ax.plot(monthly.index, monthly[region], marker='o', markersize=3, linewidth=1.5,
                color=REGION_COLORS[region], label=region)
    ax.yaxis.set_major_formatter(currency)                                    # axis format -> currency
    ax.set_title('Line_Sales_Trend')
    ax.set_xlabel('Month of Order Date')
    ax.set_ylabel('SUM(Sales)')
    ax.legend(title='Region', fontsize=8, ncol=4)


def scatter_profit_vs_sales(data, ax, by_segment=False, highlight_subcategory=None, annotate=True):
    """Worksheet 3: SUM(Sales) on Columns, SUM(Profit) on Rows, Sub-Category on Detail, Category on Color,
    optional Segment on Shape, Average reference lines on both axes."""
    keys = ['Sub-Category', 'Category'] + (['Segment'] if by_segment else [])
    agg = data.groupby(keys)[['Sales', 'Profit']].sum().reset_index()
    for _, row in agg.iterrows():
        dim = highlight_subcategory is not None and row['Sub-Category'] != highlight_subcategory
        ax.scatter(row['Sales'], row['Profit'], s=90, color=CATEGORY_COLORS[row['Category']],
                   marker=SEGMENT_MARKERS[row['Segment']] if by_segment else 'o',
                   edgecolor='black', alpha=0.2 if dim else 0.9)
        if annotate and not by_segment:
            ax.annotate(row['Sub-Category'], (row['Sales'], row['Profit']), fontsize=7,
                        xytext=(4, 4), textcoords='offset points', alpha=0.3 if dim else 1)
    ax.axvline(agg['Sales'].mean(), color='grey', linestyle='--', linewidth=1)   # Average Line (Table)
    ax.axhline(agg['Profit'].mean(), color='grey', linestyle='--', linewidth=1)
    ax.axhline(0, color='black', linewidth=0.6)
    ax.xaxis.set_major_formatter(currency_k)
    ax.yaxis.set_major_formatter(currency_k)
    ax.set_title('Scatter_Profit_vs_Sales')
    ax.set_xlabel('SUM(Sales)')
    ax.set_ylabel('SUM(Profit)')
    handles = [plt.Line2D([], [], marker='o', linestyle='', color=c, markeredgecolor='black', label=k)
               for k, c in CATEGORY_COLORS.items()]
    if by_segment:
        handles += [plt.Line2D([], [], marker=m, linestyle='', color='white', markeredgecolor='black', label=k)
                    for k, m in SEGMENT_MARKERS.items()]
    handles.append(plt.Line2D([], [], color='grey', linestyle='--', label='Average'))
    ax.legend(handles=handles, fontsize=7, loc='best')
```

Task 3: Worksheet 1 – Bar Chart (Sales by Category, coloured by Region)

```
fig, ax = plt.subplots(figsize=(10, 6))
bar_sales_by_category(df, ax)
plt.show()

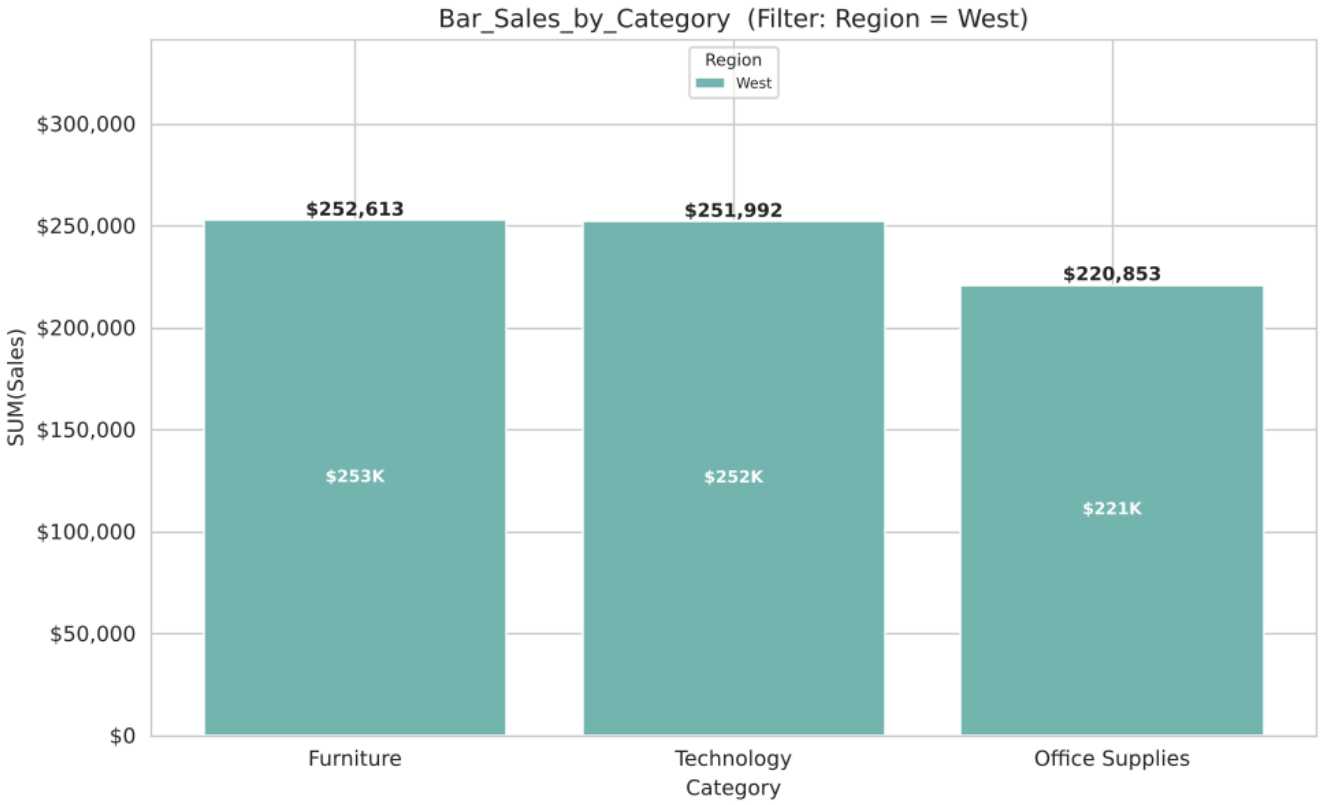
display(df.pivot_table(index='Category', columns='Region', values='Sales', aggfunc='sum', margins=True,
                       margins_name='Total').round(2))
```



Region	Central	East	South	West	Total
Category					
Furniture	163797.16	208291.20	117298.68	252612.74	741999.80
Office Supplies	167026.42	205516.06	125651.31	220853.25	719047.03
Technology	170416.31	264973.98	148771.91	251991.83	836154.03
Total	501239.89	678781.24	391721.90	725457.82	2297200.86

Optional filter – Region (equivalent to *Region → Filters → Show Filter*, here showing only the West region)

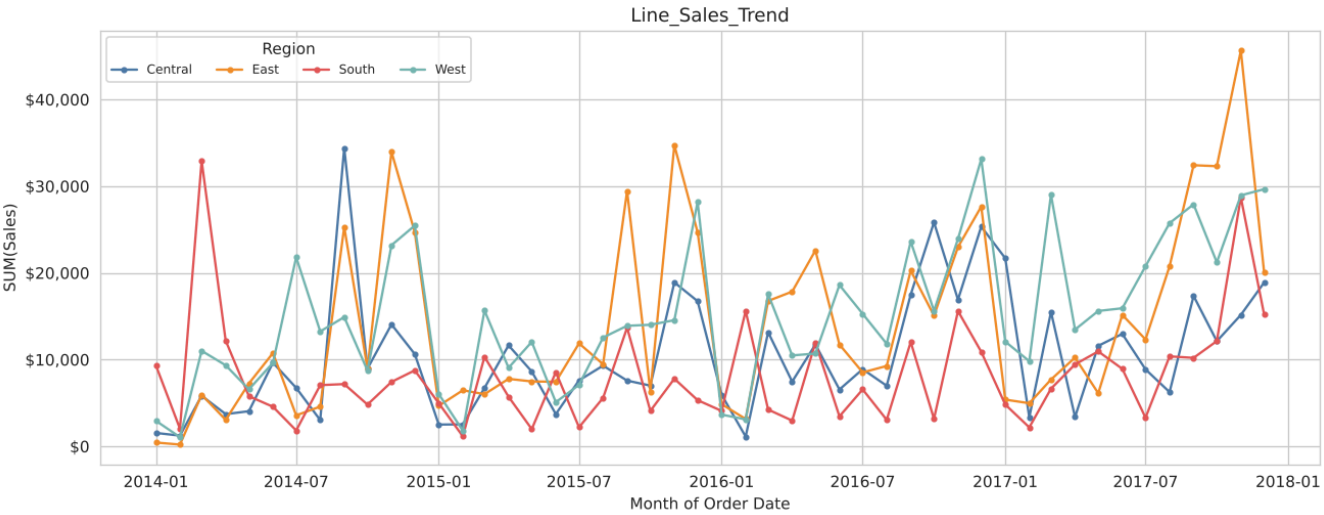
```
fig, ax = plt.subplots(figsize=(10, 6))
bar_sales_by_category(df[df['Region'] == 'West'], ax)
ax.set_title('Bar_Sales_by_Category  (Filter: Region = West)')
plt.show()
```



Task 4: Worksheet 2 – Line Chart (Monthly Sales Trend by Region)

```
fig, ax = plt.subplots(figsize=(14, 5))
line_sales_trend(df, ax)
plt.show()

yearly = df.pivot_table(index='Year', columns='Region', values='Sales', aggfunc='sum', margins=True,
                        margins_name='Total').round(2)
display(yearly)
```

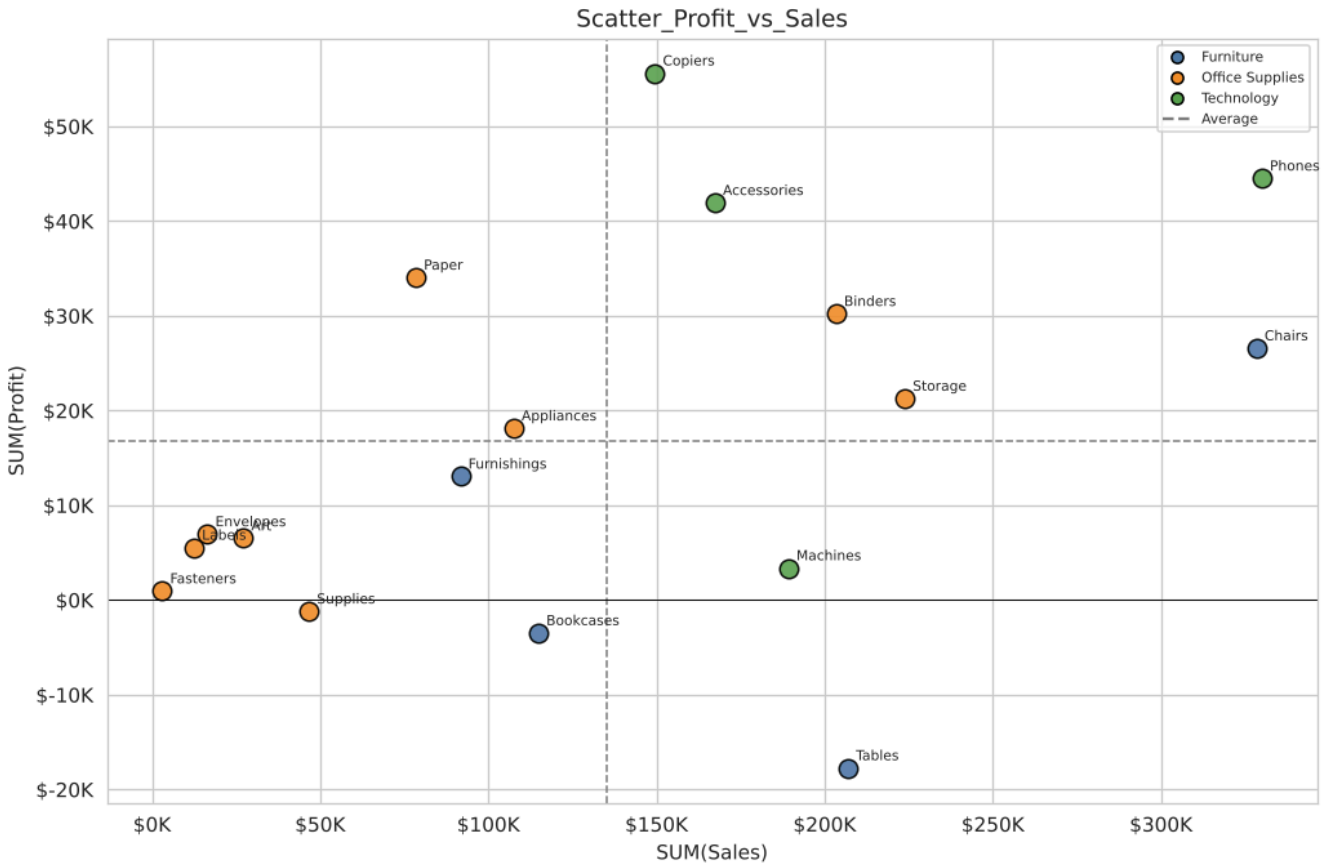


Region	Central	East	South	West	Total
Year					
2014	103838.16	128680.46	103845.84	147883.03	484247.50
2015	102874.22	156332.06	71359.98	139966.25	470532.51
2016	147429.38	180685.82	93610.22	187480.18	609205.60
2017	147098.13	213082.90	122905.86	250128.37	733215.26
Total	501239.89	678781.24	391721.90	725457.82	2297200.86

Task 5: Worksheet 3 – Scatter Plot (Profit vs Sales by Sub-Category)

```
fig, ax = plt.subplots(figsize=(11, 7))
scatter_profit_vs_sales(df, ax)
plt.show()

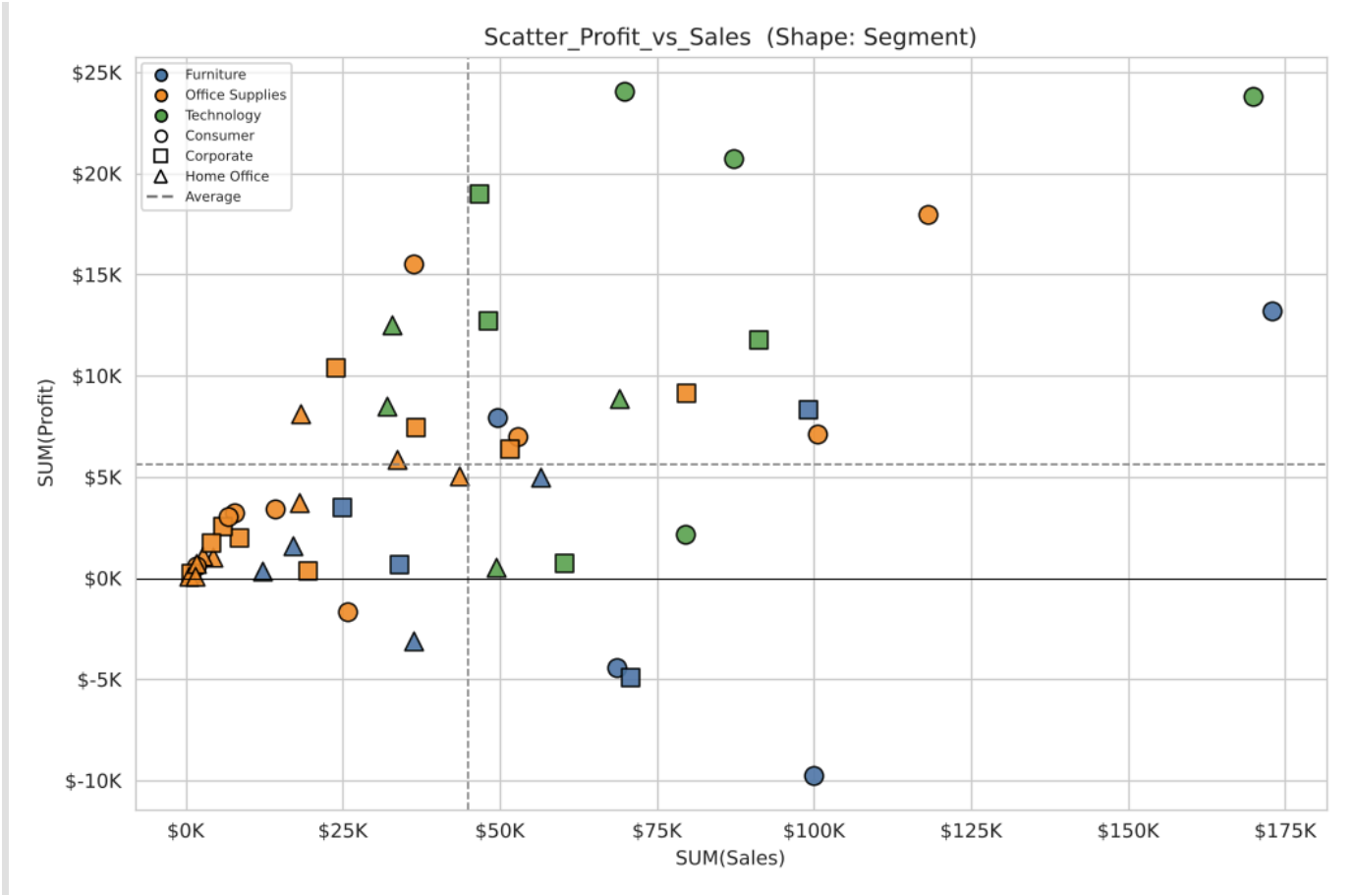
subcat = (df.groupby(['Category', 'Sub-Category'])[['Sales', 'Profit']].sum()
          .sort_values('Profit', ascending=False).round(2))
display(subcat)
```



		Sales	Profit
Category	**Sub-Category**		
Technology	**Copiers**	149528.03	55617.82
	Phones	330007.05	44515.73
	Accessories	167380.32	41936.64
Office Supplies	**Paper**	78479.21	34053.57
	Binders	203412.73	30221.76
Furniture	**Chairs**	328449.10	26590.17
Office Supplies	**Storage**	223843.61	21278.83
	Appliances	107532.16	18138.01
Furniture	**Furnishings**	91705.16	13059.14
Office Supplies	**Envelopes**	16476.40	6964.18
	Art	27118.79	6527.79
	Labels	12486.31	5546.25
Technology	**Machines**	189238.63	3384.76
Office Supplies	**Fasteners**	3024.28	949.52
	Supplies	46673.54	-1189.10
Furniture	**Bookcases**	114880.00	-3472.56
	Tables	206965.53	-17725.48

Optional: Segment on Shape + Average reference lines on both axes

```
fig, ax = plt.subplots(figsize=(11, 7))
scatter_profit_vs_sales(df, ax, by_segment=True)
ax.set_title('Scatter_Profit_vs_Sales  (Shape: Segment)')
plt.show()
```



Task 6: Build the Dashboard – "Sales and Profit Dashboard" (1200 × 800)

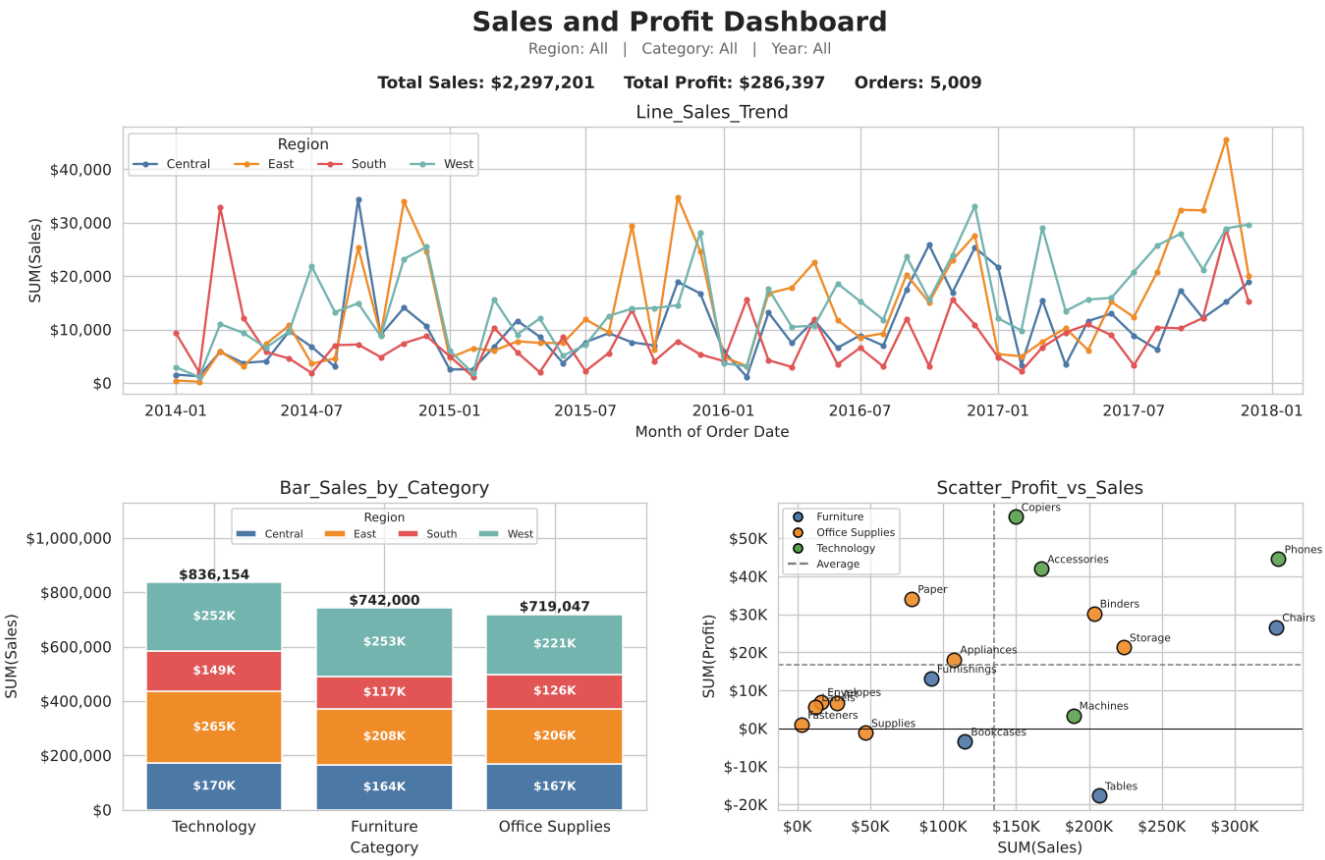
Line chart at the top, bar chart bottom-left, scatter plot bottom-right.

```
def build_dashboard(region='All', category='All', year='All', highlight_subcategory=None, title_note=''):
    data = df.copy()
    if region != 'All':
        data = data[data['Region'] == region]
    if category != 'All':
        data = data[data['Category'] == category]
    if year != 'All':
        data = data[data['Year'] == year]

    fig = plt.figure(figsize=(12, 8), dpi=100)                # 1200 x 800 pixels
    grid = fig.add_gridspec(2, 2, height_ratios=[1, 1.15], hspace=0.38, wspace=0.25)
    line_sales_trend(data, fig.add_subplot(grid[0, :]))       # top
    bar_sales_by_category(data, fig.add_subplot(grid[1, 0]))  # bottom-left
    scatter_profit_vs_sales(data, fig.add_subplot(grid[1, 1]),
                            highlight_subcategory=highlight_subcategory)   # bottom-right

    filters = f"Region: {region}   |   Category: {category}   |   Year: {year}"
    fig.suptitle('Sales and Profit Dashboard', fontsize=18, fontweight='bold', y=0.995)
    fig.text(0.5, 0.945, filters + (f"   |   {title_note}" if title_note else ''), ha='center', fontsize=10,
             color='dimgray')
    fig.text(0.5, 0.905, f"Total Sales: \\${data['Sales'].sum():,.0f}     Total Profit: \\${data['Profit'].sum():,.0f}"
             f"     Orders: {data['Order ID'].nunique():,}", ha='center', fontsize=11, fontweight='bold')
    fig.subplots_adjust(top=0.86, bottom=0.07, left=0.07, right=0.98)
    return fig

dashboard = build_dashboard()
plt.show()
```

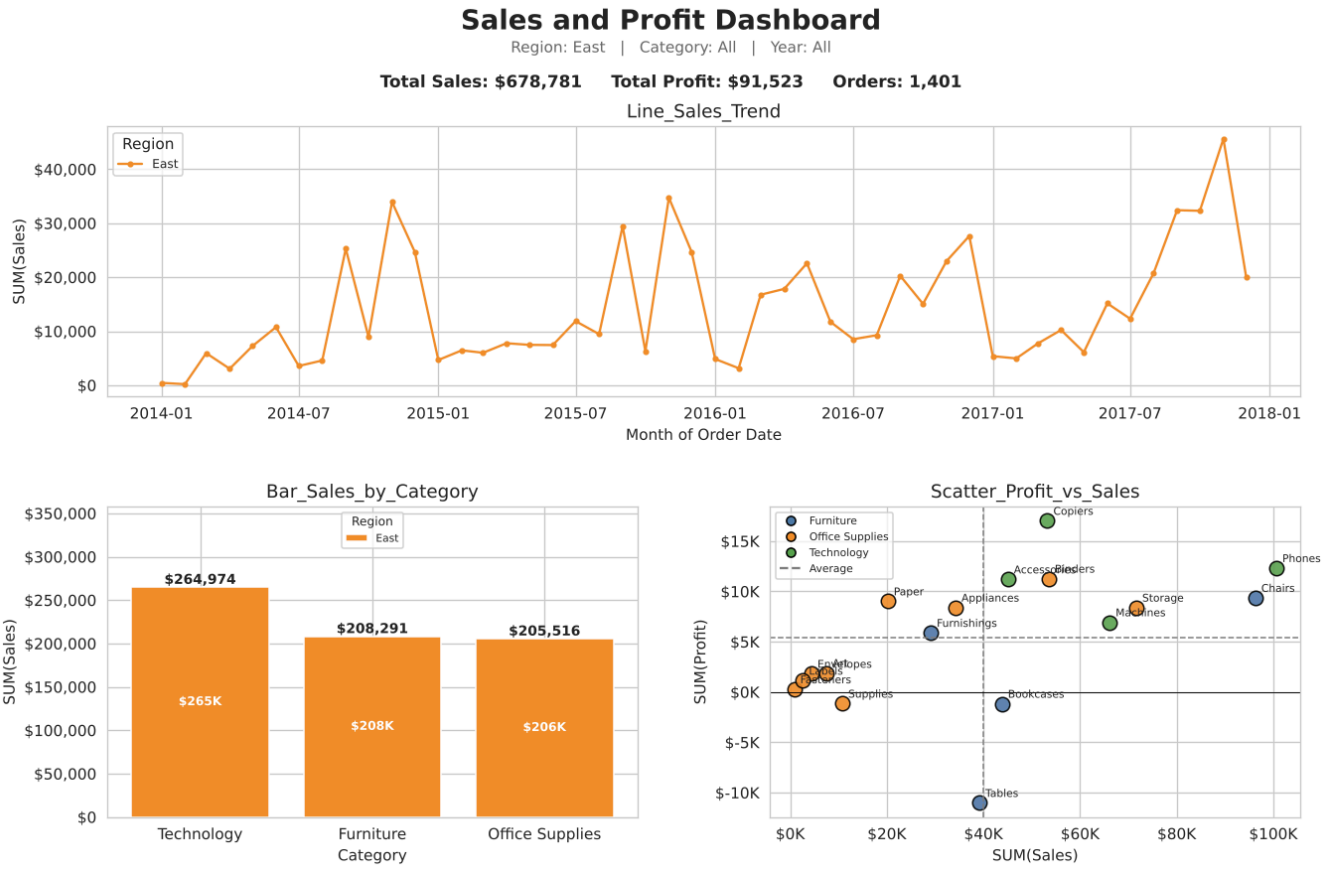


Task 7: Add Interactivity

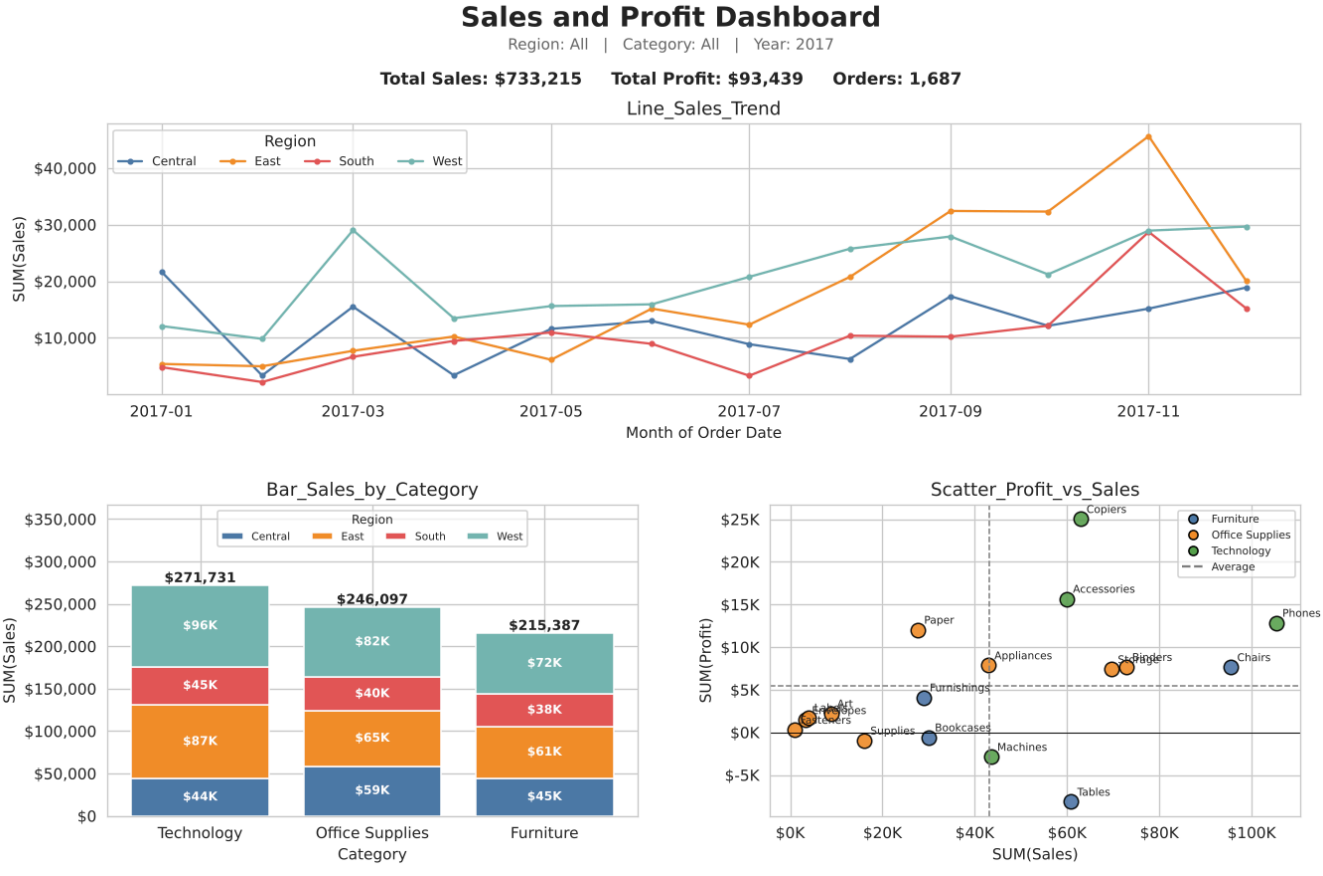
(a) Filters – Region, Category, Year

The dashboard function takes the filter values, the same as choosing a value in Tableau's filter cards.

```
build_dashboard(region='East')
plt.show()
```




```
build_dashboard(year=2017)
plt.show()
```



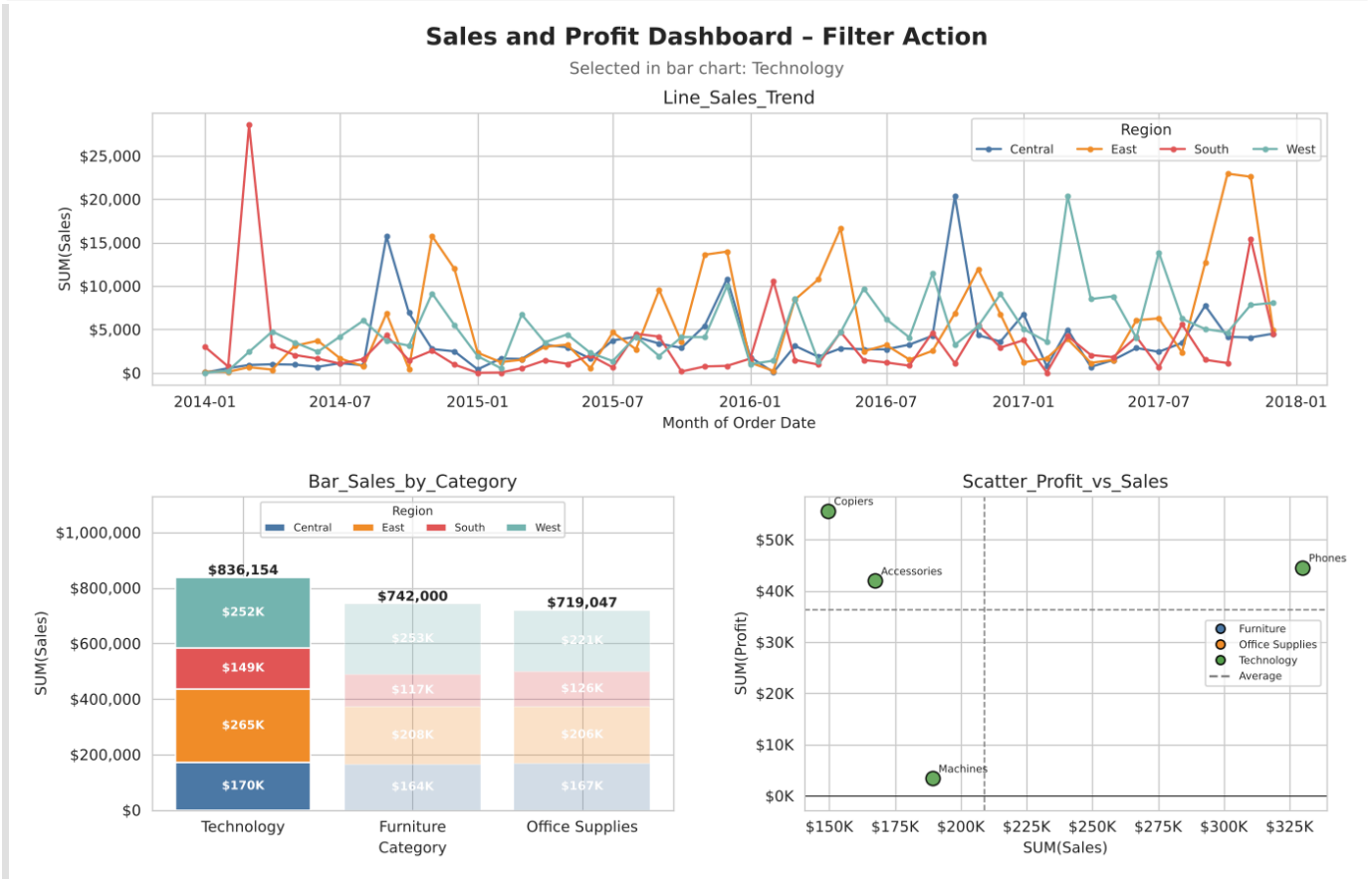
(b) Filter Action – selecting a bar (Category) filters the Line and Scatter charts

Source sheet: Bar chart · Target sheets: Line and Scatter · Run action on: Select · Clearing the selection: Show all values

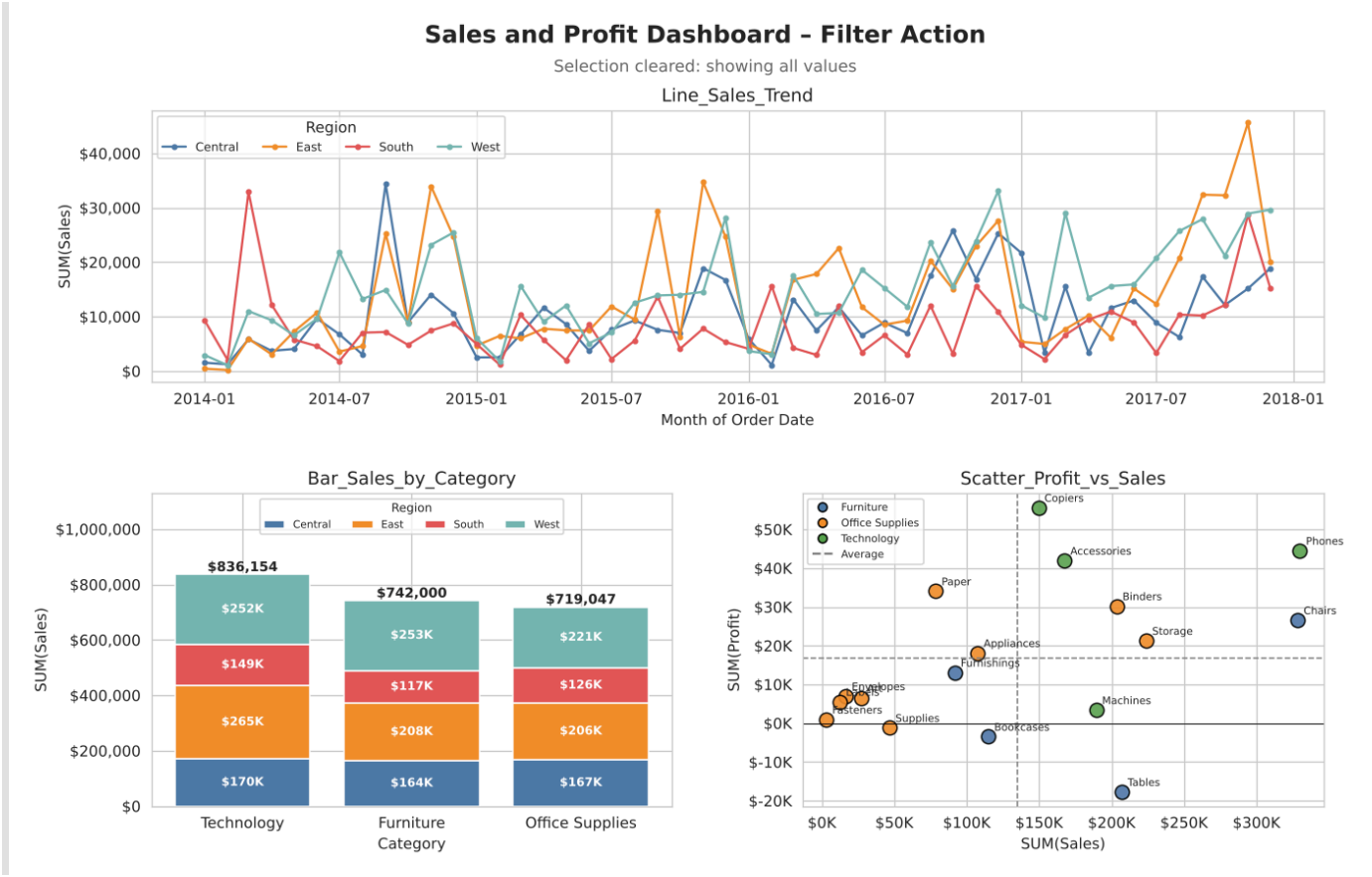
```python
def filter_action(selected_category=None):
    """Clicking a Category bar filters the other sheets; clearing the selection shows all values."""
    data = df if selected_category is None else df[df['Category'] == selected_category]
    note = f"Selected in bar chart: {selected_category}" if selected_category else "Selection cleared: showing all values"

    fig = plt.figure(figsize=(12, 8), dpi=100)
    grid = fig.add_gridspec(2, 2, height_ratios=[1, 1.15], hspace=0.38, wspace=0.25)
    line_sales_trend(data, fig.add_subplot(grid[0, :]))
    bar_sales_by_category(df, fig.add_subplot(grid[1, 0]), highlight_category=selected_category)  # source keeps all bars
    scatter_profit_vs_sales(data, fig.add_subplot(grid[1, 1]))
    fig.suptitle('Sales and Profit Dashboard — Filter Action', fontsize=16, fontweight='bold')
    fig.text(0.5, 0.925, note, ha='center', fontsize=11, color='dimgray')
    fig.subplots_adjust(top=0.88, bottom=0.07, left=0.07, right=0.98)
    return fig

filter_action('Technology')
plt.show()
```

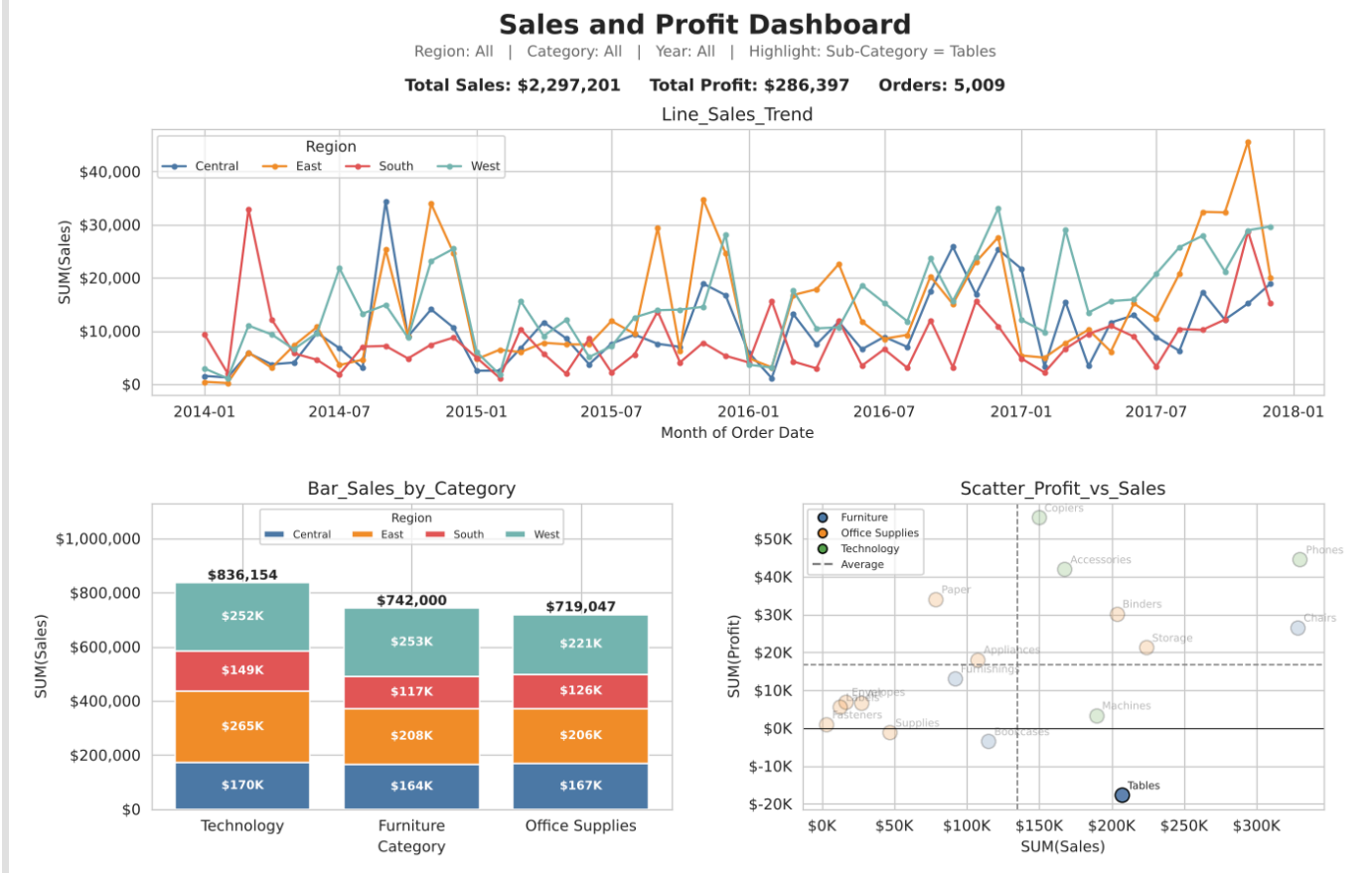



```python
filter_action(None)      # clearing the selection -> show all values
plt.show()
```



(c) Highlight Action – hovering over a Sub-Category highlights it and dims the others

```
build_dashboard(highlight_subcategory='Tables', title_note='Highlight: Sub-Category = Tables')
plt.show()
```



Task 8: Publish or Export – save the dashboard as an Image and a PDF

```
final_dashboard = build_dashboard()
final_dashboard.savefig('Sales_Dashboard.png', dpi=100, bbox_inches='tight')
final_dashboard.savefig('Sales_Dashboard.pdf', bbox_inches='tight')
plt.close(final_dashboard)

import os
for f in ['Sales_Dashboard.png', 'Sales_Dashboard.pdf']:
    print(f"Exported {f} ({os.path.getsize(f) / 1024:.1f} KB)")
```

```
Exported Sales_Dashboard.png (242.8 KB)
Exported Sales_Dashboard.pdf (46.0 KB)
```